

India DSA Trainer Job Tracker

Build & Architecture Report

Generated: 2026-09-06 17:52 UTC

Repository: github.com/yogesh889/DSA_Trainer_Online

Live page: yogesh889.github.io/DSA_Trainer_Online

Built with Claude Code (Claude Sonnet 5)

1. Objective

Automatically find "DSA Trainer" job postings in India on LinkedIn, keep a running, de-duplicated list of them, publish that list somewhere it can be checked at any time, and periodically email a PDF summary -- all without any manual searching, and without ever logging into LinkedIn or storing any LinkedIn credentials anywhere in the system.

2. Design constraint: no LinkedIn login, ever

This constraint shaped every architectural decision below, so it's worth stating up front. LinkedIn's Terms of Service prohibit automated scraping, and their systems actively detect scripted or scheduled login/browsing patterns. Automating a login for recurring, unattended scraping would risk the account being rate-limited or permanently banned, and would require storing a password or session token somewhere a scheduled job could read it -- a real security exposure. For these reasons, several requested approaches were deliberately not built:

- Logging in with a username/password to scrape authenticated pages -- declined.
- Using "Sign in with Google" as a workaround login -- declined (same underlying risk).
- Using the Claude-in-Chrome browser extension to repeatedly, automatically browse the user's own already-authenticated LinkedIn session on a recurring schedule -- declined. Even without handling a password, unattended scripted activity on a real logged-in account carries the same detection risk.
- A LinkedIn MCP/API connector -- none is connected to this account, and even LinkedIn's official API does not expose job search or "who posted this job" data outside of an enterprise Talent Solutions partnership, so a connector would not have solved this anyway.

Everything that IS built below only ever touches LinkedIn's public, logged-out pages -- the same content anyone can see without an account.

3. Architecture overview

Component	Runs on	Cadence
scrape.py	GitHub Actions	Every 5 hours
generate_report.py	GitHub Actions	~Every 5 days
index.html (live page)	GitHub Pages	Rebuilt on every push

Both scripts run inside GitHub's own cloud infrastructure via GitHub Actions -- not inside Claude's infrastructure. This matters for two reasons covered in section 6: Claude's cloud sandbox network policy blocks outbound requests to linkedin.com entirely, and GitHub Actions gives a durable schedule that keeps running indefinitely with zero Claude involvement, rather than one tied to a chat session's lifetime.

4. Data collection (scrape.py)

Source

LinkedIn's public, logged-out "guest" job-search endpoint, the same one LinkedIn itself uses to serve search-engine crawlers and anonymous visitors:

```
https://www.linkedin.com/jobs-guest/jobs/api/seeMoreJobPostings/search
?keywords=DSA+Trainer&location=India&start=<page offset>
```

Method

- Plain HTTP GET via Python's urllib, with a standard desktop browser User-Agent header.
- Paginates in pages of 10 results, up to 6 pages, stopping early once LinkedIn starts repeating results.
- A 2-second delay between page requests to avoid hammering the endpoint.
- HTML parsed with BeautifulSoup: title, company, location, "posted X ago" text, and canonical job URL.

De-duplication and freshness filter

Each listing's numeric job ID (parsed from its URL) is the dedup key. On every run, listings already being tracked have their title/company/location/posted-text refreshed from the live page (in case they changed), while a brand-new listing is recorded with a first_seen_utc timestamp. Every run also re-parses each listing's relative "posted X ago" text into an approximate age in days and drops anything outside a 1-day-to-31-day window -- too-fresh postings are skipped until they age past a day, and postings older than about a month are removed even if they were tracked before.

Output

- jobs.json -- the structured record of every currently-tracked listing.
- jobs.md -- the same data as a human-readable markdown table.
- index.html -- a static HTML page listing everything, served by GitHub Pages.

5. Hosting: the live page

GitHub Pages was enabled on the repository (branch: master, root) so index.html is served directly at:

https://yogesh889.github.io/DSA_Trainer_Online/

Because the scraper workflow regenerates and commits index.html on every run (when anything changed), the live page reflects the tracker's current state within a couple of minutes of each 5-hourly run, with no separate deploy step needed.

6. Scheduling: why GitHub Actions instead of Claude's cloud

The first version of this scraper was set up as a Claude Code "routine" -- a scheduled cloud agent session, running every 5 hours. On its first real run, every attempt to reach linkedin.com failed with an EGRESS_BLOCKED error: Claude's cloud sandbox has a network policy that blocks that domain outright, for every session, regardless of login. That routine was disabled once this was confirmed, and the scraper was rewritten as a plain Python script committed to this repository, triggered by a GitHub Actions workflow instead:

```
# .github/workflows/dsa-trainer-job-watch.yml
on:
  schedule:
    - cron: "13 */5 * * *" # every 5 hours
  workflow_dispatch: {}
```

GitHub Actions runners have normal outbound internet access, so the same scraping logic that worked from a local machine works there unmodified. This also means the tracker keeps running even if this Claude Code session, or the user's own computer, is closed -- it lives entirely inside the GitHub repository's own infrastructure.

7. Email reporting (generate_report.py)

A second workflow, dsa-trainer-email-report, runs on an approximately-every-5-day schedule (cron "0 3 */5 * *" -- day-of-month 1/6/11/16/21/26/31, which approximates every 5 days but has a shorter gap at some month boundaries;

standard cron cannot express a true rolling 5-day interval across months).

What it does

- Reads jobs.json and report_state.json (which tracks the last report's timestamp).
- Builds a multi-page PDF (via the fpdf2 library) listing new listings since the last report, or the full list if there are none.
- Emails that PDF via Gmail SMTP (smtp.gmail.com:465) to the configured recipient.
- Updates report_state.json and commits it, so the next run only reports what's new.

Credentials

This step needs a Gmail account to send from. Per the same no-credential-handling principle as section 2, the repository owner creates a Gmail App Password themselves and adds it as a GitHub Actions repository secret (SMTP_USER / SMTP_PASS) directly in GitHub's own settings UI -- Claude never sees or handles that credential at any point.

8. Investigated separately: job-poster profile links

A recurring request during this project was to also capture the LinkedIn profile of whoever posted each job, so it could be added to the tracker. This was investigated thoroughly, using the Claude-in-Chrome browser extension connected to the user's own, already-authenticated LinkedIn session (a one-time, interactive, user-supervised check -- not automation):

- Confirmed the public/guest version of a job page contains zero profile links or hiring-team data at all -- not a parsing gap, the server simply omits it for anonymous requests.
- Confirmed, live and logged in, that LinkedIn only shows a "Meet the hiring team" card when someone manually tagged themselves as the poster -- most institutional postings (universities, companies via their careers page or an ATS) don't have this.
- Manually checked 15 of the 69 tracked listings this way and found 2 with a real tagged poster (with name and profile URL); the other 13 had none.

This lookup was not wired into the automated pipeline, and will not be: doing it on a schedule would mean scripted, unattended browsing of the user's real logged-in LinkedIn session, which runs into the same account-risk concern as section 2. It remains available as a manual, one-off action on request, run interactively with the user present.

9. Setup checklist

Step	Status
Repo created, scraper + workflow pushed	Done
GitHub Pages enabled	Done
Job-watch workflow verified via a real run	Done
Gmail App Password + SMTP_USER/SMTP_PASS secrets	Owner action required

10. Known limitations

- Freshness window (1-31 days) is recalculated from LinkedIn's own relative text each run, which is only as precise as that text (e.g. "1 month ago" is treated as 30 days).
- The 5-day email cron is a calendar approximation, not a true rolling interval; it can occasionally fire 1-4 days early at a month boundary.

- LinkedIn's guest search endpoint returns a limited result depth (roughly 40-60 postings) before it starts repeating results, so extremely long-tail postings beyond that may not surface.
- Job-poster identity is available for only a small minority of listings, and is not tracked automatically (see section 8).
- LinkedIn's normal post/feed search (people posting about hiring outside the structured Jobs section) requires a login for guests and is out of scope for the same reason as section 2.

11. File manifest

File	Purpose
scrape.py	Fetch, filter, dedupe, write outputs
generate_report.py	Build PDF + send email report
jobs.json / jobs.md	Tracked listings, machine + human readable
index.html	Live page (GitHub Pages)
report_state.json	Last-emailed-report timestamp
.github/workflows/*.yml	Scheduling